

Berliner Hochschule für Technik

Bachelorarbeit

Konzeption und Implementierung einer interaktiven Visualisierung präemptiver Scheduling-Verfahren für die Lehre

Marten Kirschenhofer
Matrikelnummer: 833825
Studiengang: Medieninformatik

Betreuung: Prof. Dr. Christian Forler
Abgabedatum: 15.09.2026

Vorwort

Diese Bachelorarbeit entstand im Studiengang Medieninformatik an der Berliner Hochschule für Technik. Sie behandelt die Konzeption und Implementierung einer interaktiven Anwendung zur Veranschaulichung präemptiver CPU-Scheduling-Verfahren.

Danksagung

Ich widme diese Arbeit meinen lieben Schwiegereltern, die mir stets mit gutem Rat zur Seite standen, meiner wundervollen Verlobten, die mich auch in schwierigen Zeiten unterstützt hat, und meinem besten Freund, bei dem ich trotz des großen Stresses stets ich selbst sein konnte.

Inhaltsverzeichnis

Vorwort	ii
Abbildungsverzeichnis	vi
Tabellenverzeichnis	vii
Verzeichnis der Listings	viii
1. Einleitung	1
1.1. Motivation und Problemstellung	1
1.2. Zielsetzung der Arbeit	1
1.3. Thematische Ausrichtung und Abgrenzung	2
1.4. Vorgehensweise und Aufbau der Arbeit	2
1.5. Leitgedanke der Arbeit	2
2. Grundlagen	3
2.1. Betriebssysteme und Prozessverwaltung	3
2.2. CPU-Scheduling in Betriebssystemen	5
2.3. Präemptive versus nicht-präemptive Scheduling-Verfahren	6
2.4. Bewertungsmetriken	6
2.5. Scheduling-Verfahren	8
2.6. Vergleich der theoretischen Perspektiven	10
2.7. Algorithmusvisualisierung als Vermittlungsform	11
2.8. Accessibility als fachliche und gestalterische Grundlage	11
2.9. Zwischenfazit	11
3. Anforderungsanalyse	13
3.1. Zielgruppe und Einsatzkontext	13
3.2. Fachliche Anforderungen	13
3.3. Didaktische Anforderungen	13
3.4. Interaktions- und Usability-Anforderungen	14
3.5. Anforderungen an Accessibility	14
3.6. Technische Qualitätsziele	15
3.7. Abgeleitete funktionale und nicht-funktionale Anforderungen	16
4. Systementwurf	17
4.1. Entwurfsziele und Leitprinzipien	17
4.2. Gesamtarchitektur und Schichtenmodell	17
4.3. Komponenten und Verantwortlichkeiten	19
4.4. Datenfluss und Zustandsmodell	20
4.5. Interaktionsmodell für den Lernprozess	20
4.6. HCI-Leitlinien im Entwurf	21
4.7. Accessibility als Entwurfsprinzip	21

4.8. Erweiterbarkeit und Entwurfsgrenzen	22
5. Implementierung	23
5.1. Technische Grundlagen und verwendete Technologien	23
5.2. Datenmodell und Simulationskern	24
5.3. Umsetzung der Scheduling-Verfahren	26
5.4. Umsetzung der Visualisierung	28
5.5. Interaktionsmechanismen und Nutzerworkflow	29
5.6. Umsetzung der Accessibility-Anforderungen	29
5.7. Build- und Deploymentprozess	30
5.8. Besondere technische Herausforderungen	30
6. Qualitätssicherung	31
6.1. Qualitätsziele und Prüfstrategie	31
6.2. Automatisierte Verifikation der Simulationslogik	31
6.3. Prüfung von Interaktion und Accessibility	32
6.4. Reproduzierbarkeit und Vergleichbarkeit	32
6.5. Erreichte Testabdeckung und Grenzen	33
6.6. Fazit zur Qualitätssicherung	33
7. Ergebnis und Produktpräsentation	34
7.1. Entwickeltes Endprodukt	34
7.2. Produktüberblick und Nutzerfluss	34
7.3. Produktpräsentation mit Screenshots	35
7.4. Umgesetzte Funktionalität im Soll-Ist-Abgleich	35
7.5. Accessibility-relevante Produktzustände	37
7.6. Ergebnisbetrachtung	37
8. Fazit und Ausblick	39
8.1. Ausblick	39
Literaturverzeichnis	40
A. Anhang	41
A.1. Ergänzende Abbildungen	41
A.2. Zusätzliche Testfälle und Szenarien	41
A.3. Operationalisierungsmatrix Accessibility	41
A.4. Kompaktes manuelles Accessibility-Prüfprotokoll	42
A.5. Ergänzende technische Artefakte (z. B. Konfigurationsbeispiele)	42

Abbildungsverzeichnis

4.1. Komponentenmodell der Anwendung mit Präsentationsschicht, Composables, Simulationskern und Domänenmodell.	18
4.2. Übersicht der Benutzeroberfläche: Eingabebereich, Simulationsfenster und Metriken-Anzeige.	19
5.1. Domänenmodell des Simulationslaufs mit seinen Ergebnisstrukturen.	25
5.2. Sequenzdiagramm eines Simulationslaufs von der Benutzeraktion bis zur Aktualisierung der Benutzeroberfläche.	27
5.3. Zustände und Übergänge eines Prozesses im Simulationsmodell.	28
7.1. Startansicht der Anwendung mit Ausgangszustand und Einstieg in die Szenariokonfiguration.	35
7.2. Geladenes Szenario mit sichtbaren Task-Parametern als Grundlage für die Simulation.	35
7.3. Fokusansicht mit zeitlicher Ablaufdarstellung und Kennzahlenbereich für einen einzelnen Lauf.	36
7.4. Detaildarstellung der Gantt-Ansicht mit segmentierter Ausführung und sichtbaren Task-Wechseln.	36
7.5. Warteschlangen- und Statusdarstellung zur Einordnung der aktuellen Scheduling-Entscheidung.	36
7.6. Kennzahlenbereich mit Wartezeit, Durchlaufzeit und Reaktionszeit als quantitative Auswertung des Laufs.	36
7.7. Vergleichsansicht mehrerer Verfahren auf identischer Szenariobasis.	37

Tabellenverzeichnis

2.1. Für die Simulation verwendete Prozesszustände.	4
2.2. Beispielszenario zur Gegenüberstellung der Scheduling-Verfahren.	8
2.3. Ergänzende Perspektiven der verwendeten Betriebssystemliteratur.	10
3.1. Operationalisierte Accessibility-Anforderungen des fokussierten Prüfstands.	15

Verzeichnis der Listings

1. Einleitung

1.1. Motivation und Problemstellung

Scheduling-Verfahren gehören zu den Kernmechanismen moderner Betriebssysteme. Sie legen fest, welcher Prozess zu welchem Zeitpunkt CPU-Zeit erhält, und beeinflussen damit Reaktionsfähigkeit, Fairness und Ressourcenauslastung Stallings [2003], Tanenbaum and Bos [2016]. Obwohl die Verfahren in der Lehre formal behandelt werden, bleibt ihr zeitlicher Ablauf für Studierende häufig abstrakt.

Dies gilt insbesondere für präemptive Verfahren: Häufige Unterbrechungen, Wiedereinplanungen und Zustandswechsel erschweren die mentale Nachvollziehbarkeit. Der Unterschied zwischen Verfahren ist theoretisch beschreibbar, wird jedoch in seiner konkreten Wirkung auf Ablauf, Wartezeit und Reaktionszeit oft erst durch eine zeitliche Darstellung verständlich.

Vor diesem Hintergrund wurde eine interaktive Webanwendung entwickelt, die präemptive Scheduling-Verfahren als steuerbaren Ablauf visualisiert. Die Arbeit zielt auf die Entwicklung eines didaktisch nutzbaren Softwaresystems; sie verfolgt keine hypothesengetriebene Wirksamkeitsstudie.

1.2. Zielsetzung der Arbeit

Ziel der Arbeit ist die Konzeption und Implementierung einer interaktiven Webanwendung zur Veranschaulichung präemptiver Scheduling-Verfahren. Die Anwendung soll:

- ausgewählte Verfahren korrekt simulieren,
- deren zeitliche Dynamik verständlich visualisieren,
- Verfahren auf identischer Szenariobasis vergleichbar machen,
- und im Lehrkontext praktisch einsetzbar sein.

Die Arbeit verbindet hierfür Betriebssystemgrundlagen mit Prinzipien des Software Engineering und der Human-Computer Interaction. Im Zentrum stehen eine konsistente Architektur, transparente Interaktion und ein belastbares Produktartefakt.

1.3. Thematische Ausrichtung und Abgrenzung

Die Arbeit ist als engineering-orientierte Produktdokumentation angelegt. Gegenstand ist die Entwicklung und Begründung eines konkreten Artefakts inklusive fachlicher, technischer und gestalterischer Entscheidungen.

Abgegrenzt wird sie von empirischen Wirksamkeitsstudien mit Nutzergruppen, statistischer Auswertung und Hypothesentests. Im Fokus stehen stattdessen Simulationskonsistenz, Architekturentscheidungen, Implementierung und qualitätsgesicherte Produktdarstellung.

1.4. Vorgehensweise und Aufbau der Arbeit

Die Vorgehensweise umfasst sechs Schritte:

1. fachliche Grundlagen zu CPU-Scheduling und Bewertungsmetriken,
2. Ableitung funktionaler und nicht-funktionaler Anforderungen,
3. Systementwurf mit Architektur- und Interaktionsmodell,
4. Implementierung der Webanwendung,
5. Qualitätssicherung von Simulationslogik und Interaktion,
6. Ergebnisdarstellung und Einordnung des Produkts.

Kapitel 2 behandelt die Grundlagen, Kapitel 3 die Anforderungsanalyse, Kapitel 4 den Systementwurf, Kapitel 5 die Implementierung und Kapitel 6 die Qualitätssicherung. Kapitel 7 präsentiert das entwickelte Produkt; Kapitel 8 schließt mit Fazit und Ausblick.

1.5. Leitgedanke der Arbeit

Der leitende Gedanke lautet:

Praktische Verständlichkeit entsteht durch das Zusammenspiel aus fachlicher Korrektheit, klarer Architektur, transparenter Interaktion und nachvollziehbarer Ablaufdarstellung.

Damit wird Scheduling-Visualisierung nicht nur als Implementierungsaufgabe verstanden, sondern als Verbindung von Fachmodell, Interaktionsgestaltung und didaktischer Vermittlung.

2. Grundlagen

Dieses Kapitel stellt die theoretischen Grundlagen dar, auf denen Konzeption, Implementierung und Bewertung der Anwendung basieren.

2.1. Betriebssysteme und Prozessverwaltung

Betriebssysteme koordinieren den Zugriff auf Hardware-Ressourcen und stellen Ausführungsumgebungen für Programme bereit. Zu ihren Kernaufgaben zählt die Verwaltung konkurrierender Ausführungseinheiten, die um CPU-Zeit, Speicher und weitere Ressourcen konkurrieren Stallings [2003], Tanenbaum and Bos [2016].

Ein Prozess bezeichnet dabei eine isolierte Ausführungsumgebung mit eigenem Adressraum und eigenem Ressourcenzustand. Prozesse durchlaufen typischerweise Zustände wie *bereit*, *laufend* und *blockiert*. Das Betriebssystem steuert die Übergänge zwischen diesen Zuständen und trifft Scheduling-Entscheidungen über die nächste CPU-Zuteilung.

Ergänzend existiert die Thread-Abstraktion als leichtgewichtige Ausführungseinheit innerhalb eines Prozesses. Threads teilen sich den Adressraum und weitere Ressourcen des Prozesses; Kontextwechsel zwischen Threads sind daher in der Regel günstiger als zwischen Prozessen. Für die vorliegende Arbeit ist jedoch die Prozessabstraktion ausreichend, da die zentralen Scheduling-Prinzipien unverändert gelten.

2.1.1. Aufgaben des Betriebssystems

Die Betriebssystemfunktionen lassen sich unter anderem in Prozessverwaltung, Speicher-verwaltung, Ein-/Ausgabeverwaltung und Dateiverwaltung gliedern. Für das CPU-Scheduling sind vor allem Prozessverwaltung und Unterbrechungsbehandlung relevant. Das Betriebssystem verwaltet Prozesskontexte, führt Zustandswechsel durch und reagiert auf Ereignisse wie Timer-Interrupts, Ein-/Ausgabe-Abschlüsse oder die Ankunft neuer Prozesse.

Stallings beschreibt Betriebssysteme dabei stark aus der Perspektive der Verwaltung und Zuteilung von Ressourcen. Tanenbaum und Bos ordnen dieselben Aufgaben stärker in ein konzeptionelles Modell aus Prozessen, Threads, Zuständen und Betriebssystemkomponenten ein. Für diese Arbeit werden beide Perspektiven verbunden: Die Ressourcenperspektive erklärt, warum Scheduling erforderlich ist, während das Prozessmodell beschreibt, wie die daraus entstehenden Entscheidungen ablaufen Stallings [2003], Tanenbaum and Bos [2016].

2.1.2. Prozesszustände und Zustandsübergänge

Ein Prozess befindet sich zu jedem Zeitpunkt in einem bestimmten Ausführungszustand. Für die Modellierung des Visualizers werden die Zustände *pending*, *ready*, *running*, *preempted* und *finished* verwendet. Ein neu angelegter Prozess ist zunächst noch nicht bereit, solange seine Ankunftszeit nicht erreicht ist. Danach kann er in die Bereitschaftsmenge aufgenommen werden. Der Zustand *running* bezeichnet den Prozess, dem aktuell CPU-Zeit zugewiesen ist.

Wird ein laufender Prozess durch einen anderen verdrängt, wechselt er in einen unterbrochenen beziehungsweise wieder bereiten Zustand. Nach einer erneuten Auswahl kann er weiterlaufen. Mit dem Abschluss der benötigten Burst-Zeit erreicht er den Zustand *finished*. Diese Übergänge bilden die fachliche Grundlage für die Ereignisse des Simulationslaufs und für die Segmentierung der Gantt-Ansicht.

Tabelle 2.1.: Für die Simulation verwendete Prozesszustände.

Zustand	Bedeutung	Typischer Übergang
<i>pending</i>	Prozess ist noch nicht angekommen.	Ankunftszeit erreicht → <i>ready</i>
<i>ready</i>	Prozess wartet auf CPU-Zuteilung.	Auswahl durch Scheduler → <i>running</i>
<i>running</i>	Prozess verwendet die CPU.	Abschluss, Präemption oder Quantumablauf
<i>preempted</i>	Prozess wurde unterbrochen und wartet auf erneute Auswahl.	erneute Auswahl → <i>running</i>
<i>finished</i>	Prozess hat seine benötigte Ausführungszeit verbraucht.	Endzustand

2.1.3. Scheduler, Dispatcher und Kontextwechsel

Der Scheduler entscheidet, welcher bereite Prozess als Nächstes CPU-Zeit erhalten soll. Der Dispatcher setzt diese Entscheidung technisch um, indem er den Kontext des bisher laufenden Prozesses sichert und den Kontext des ausgewählten Prozesses lädt. Zum Prozesskontext gehören je nach System unter anderem Registerinhalte, Program Counter und Verwaltungsinformationen.

Der Wechsel zwischen zwei Prozessen wird als Kontextwechsel bezeichnet. Er ermöglicht Präemption, verursacht aber Verwaltungsaufwand, weil der laufende Zustand gesichert und ein anderer Zustand wiederhergestellt werden muss. In der Simulation wird dieser Aufwand nicht als reale Prozessorzeit modelliert; die Anzahl der Kontextwechsel wird jedoch als Kennzahl erfasst. Dadurch kann sichtbar gemacht werden, dass ein Verfahren zwar eine kurze Reaktionszeit erreichen, zugleich aber mehr Wechsel erzeugen kann.

Ein Scheduling-Ereignis kann durch mehrere Ursachen ausgelöst werden. Dazu gehören die Ankunft eines neuen Prozesses, der Ablauf eines Zeitquantums, die Beendigung ei-

nes Prozesses oder ein Wechsel von einer blockierten in eine bereite Situation. Die Implementierung abstrahiert diese Betriebssystemereignisse auf diskrete Zeitschritte. Diese Abstraktion ist für eine didaktische Visualisierung geeignet, weil sie jeden relevanten Zustandswechsel explizit darstellen kann.

2.2. CPU-Scheduling in Betriebssystemen

CPU-Scheduling beschreibt die Auswahlstrategie zur Vergabe von Prozessorzeit an bereite Ausführungseinheiten. Ziel ist ein Ausgleich zwischen konkurrierenden Kriterien wie Reaktionszeit, Wartezeit, Auslastung und Fairness Stallings [2003].

In Mehrprogrammsystemen erfolgt diese Auswahl unter variierenden Ankunftszeiten und Laufzeiten. Scheduling-Verfahren werden daher üblicherweise als Heuristiken formuliert, deren Eignung vom Lastprofil und den Systemzielen abhängt. Zentrale Fragestellungen sind die Auswahl des nächsten Prozesses, der Umgang mit Präemption und Kontextwechseln sowie die Bewertung über geeignete Kennzahlen.

2.2.1. Eingabeparameter und Entscheidungsgrundlage

Ein Scheduling-Szenario besteht im Kern aus einer Menge von Prozessen. Für jeden Prozess werden mindestens eine Ankunftszeit und eine benötigte Ausführungszeit, die sogenannte Burst-Zeit, benötigt. Je nach Verfahren kommen Priorität, Zeitquantum oder eine Zugehörigkeit zu einer Warteschlange hinzu. Die Ankunftszeit legt fest, ab welchem Zeitpunkt ein Prozess berücksichtigt werden darf. Die Burst-Zeit beschreibt, wie viel CPU-Zeit bis zum Abschluss noch benötigt wird.

Für jeden Zeitschritt wird zunächst bestimmt, welche Prozesse angekommen und noch nicht abgeschlossen sind. Aus dieser Bereitschaftsmenge wählt der Scheduler anhand der jeweiligen Verfahrensregel einen Prozess. Anschließend wird dessen Restzeit reduziert und das entsprechende Ereignis beziehungsweise Segment im Ergebnisobjekt gespeichert. Diese Reihenfolge ist wichtig, weil ansonsten Ankunft, Präemption und Abschluss in einer mehrdeutigen Reihenfolge verarbeitet werden könnten.

2.2.2. Ziele und Zielkonflikte

Scheduling-Verfahren optimieren unterschiedliche Ziele, die nicht gleichzeitig maximal erfüllt werden können. Eine kurze durchschnittliche Wartezeit kann beispielsweise zu einer langen Wartezeit einzelner Prozesse führen. Ein hoher Durchsatz kann mit einer schlechteren Reaktionszeit interaktiver Prozesse einhergehen. Häufige Präemption kann die Reaktionsfähigkeit verbessern, erhöht aber die Zahl der Kontextwechsel.

Für die Bewertung wird deshalb kein einzelner universeller Gütwert vorausgesetzt. Stattdessen werden mehrere Kennzahlen gemeinsam betrachtet. Die Vergleichsansicht der An-

wendung erlaubt zusätzlich, die Gewichtung dieser Kennzahlen an einen Anwendungsfall wie Interaktion, Batch-Verarbeitung oder Soft Real-Time anzupassen.

2.3. Präemptive versus nicht-präemptive Scheduling-Verfahren

Scheduling-Verfahren lassen sich grundlegend in präemptive und nicht-präemptive Strategien einteilen. Diese Unterscheidung beeinflusst sowohl das Laufzeitverhalten als auch die analytische Komplexität.

2.3.1. Nicht-präemptive Verfahren

Nicht-präemptive Verfahren erlauben einem Prozess, die CPU bis zum Abschluss oder bis zur Blockierung zu behalten. Entscheidungen treten dadurch nur an diskreten Punkten auf. Vorteile sind geringer Kontextwechselaufwand und einfache Ablaufstrukturen; nachteilig sind potenziell lange Wartezeiten. Typische Vertreter sind FCFS und nicht-präemptives SJF.

2.3.2. Präemptive Verfahren

Präemptive Verfahren erlauben dem Scheduler, einen laufenden Prozess aktiv zu unterbrechen und einen anderen einzuplanen. Auslöser sind typischerweise Zeitquantumsgrenzen (Timer-Interrupt) oder die Ankunft höher priorisierter Prozesse. Dadurch steigen Reaktionsfähigkeit und Steuerbarkeit, zugleich aber auch die Anzahl der Kontextwechsel Tanenbaum and Bos [2016].

2.4. Bewertungsmetriken

Die Bewertung eines Scheduling-Verfahrens benötigt messbare Größen, die aus dem Ablauf eines Simulationslaufs abgeleitet werden. Im Folgenden bezeichnet a_i die Ankunftszeit eines Prozesses i , c_i seine Fertigstellungszeit, s_i den Zeitpunkt seiner ersten Ausführung und b_i seine gesamte benötigte CPU-Zeit.

2.4.1. Durchlaufzeit

Die Durchlaufzeit beschreibt die Zeitspanne vom Eintreffen bis zum Abschluss eines Prozesses:

$$T_i = c_i - a_i \tag{2.1}$$

Sie umfasst sowohl die tatsächliche Ausführungszeit als auch Wartezeiten und Unterbrechungen. Die durchschnittliche Durchlaufzeit ergibt sich aus:

$$\bar{T} = \frac{1}{n} \sum_{i=1}^n T_i. \quad (2.2)$$

Eine geringe durchschnittliche Durchlaufzeit ist besonders für abgeschlossene Aufträge und Batch-Szenarien relevant. Sie beschreibt jedoch nicht, wie früh ein Prozess erstmals reagieren kann.

2.4.2. Wartezeit

Die Wartezeit bezeichnet die Zeit, in der ein Prozess bereit ist, aber keine CPU erhält. Bei einem Prozess ohne Blockierung kann sie aus Durchlaufzeit und Burst-Zeit berechnet werden:

$$W_i = T_i - b_i. \quad (2.3)$$

Bei präemptiven Verfahren enthält diese Größe alle Zeitanteile zwischen einzelnen Ausführungsabschnitten. Eine niedrige durchschnittliche Wartezeit kann daher anzeigen, dass die Bereitschaftsmenge insgesamt effizient bedient wurde. Zusätzlich ist die maximale Wartezeit relevant, weil ein guter Mittelwert lange Benachteiligungen einzelner Prozesse verdecken kann.

2.4.3. Reaktionszeit

Die Reaktionszeit misst, wie lange ein Prozess nach seiner Ankunft bis zur ersten CPU-Zuteilung warten muss:

$$R_i = s_i - a_i. \quad (2.4)$$

Sie ist besonders für interaktive Systeme bedeutsam. Ein Verfahren kann eine akzeptable Durchlaufzeit erreichen, aber trotzdem eine schlechte Reaktionszeit aufweisen, wenn bestimmte Prozesse erst spät erstmals ausgeführt werden.

2.4.4. Durchsatz und Kontextwechsel

Der Durchsatz beschreibt die Zahl abgeschlossener Prozesse pro Zeiteinheit. Für einen Beobachtungszeitraum von t_0 bis t_1 kann er näherungsweise als Verhältnis der abgeschlossenen Prozesse zur Zeitspanne angegeben werden:

$$X = \frac{N_{\text{fertig}}}{t_1 - t_0}. \quad (2.5)$$

Die Anzahl der Kontextwechsel ist keine reine Zeitmetrik, sondern ein Indikator für die Häufigkeit von Scheduling-Entscheidungen. Zusammen mit Präemptionsanzahl und Reaktionszeit hilft sie, den Zielkonflikt zwischen Responsivität und Verwaltungsaufwand zu beschreiben.

2.5. Scheduling-Verfahren

Die folgenden Verfahren werden im Visualizer als deterministische Strategien auf derselben Prozess- und Ereignisstruktur umgesetzt. Zur Einordnung dient ein gemeinsames Beispielszenario:

Tabelle 2.2.: Beispielszenario zur Gegenüberstellung der Scheduling-Verfahren.

Prozess	Ankunft	Burst	Priorität	Quantum
P1	0	5	2	2
P2	1	3	1	2
P3	2	2	3	2

Das Szenario ist kein vollständiger empirischer Leistungsvergleich. Es dient dazu, die unterschiedlichen Auswahlregeln und ihre sichtbaren Folgen zu erklären. Je nach Prioritätskonvention muss für das konkrete Verfahren zusätzlich festgelegt werden, ob ein kleiner oder großer Zahlenwert eine höhere Priorität bedeutet.

2.5.1. First-Come, First-Served und Shortest Job First als Referenz

First-Come, First-Served (FCFS) ordnet bereite Prozesse nach ihrer Ankunft. Der zuerst angekommene Prozess wird zuerst ausgeführt. Das Verfahren ist einfach nachvollziehbar und benötigt keine komplexe Auswahlregel, kann jedoch zu langen Wartezeiten führen, wenn ein langer Prozess vor mehreren kurzen Prozessen steht.

Shortest Job First (SJF) wählt den Prozess mit der kürzesten erwarteten beziehungsweise verbleibenden Ausführungszeit. In der nicht-präemptiven Variante bleibt der ausgewählte Prozess bis zum Abschluss aktiv. SJF dient in dieser Arbeit als Referenz für das Spannungsfeld zwischen durchschnittlicher Wartezeit und der notwendigen Kenntnis zukünftiger Ausführungszeiten. Die produktive Anwendung konzentriert sich dagegen auf präemptive Verfahren.

2.5.2. Round Robin

Round Robin verwendet eine zyklische Bereitschaftwarteschlange und ein festgelegtes Zeitquantum. Ein Prozess erhält höchstens ein Quantum und wird bei weiter bestehender Restzeit am Ende der Warteschlange erneut eingereiht. Dadurch erhalten bereite Prozesse wiederholt CPU-Zeit.

Ein kleines Quantum verbessert typischerweise die Reaktionsmöglichkeit, erzeugt aber mehr Kontextwechsel. Ein großes Quantum reduziert den Verwaltungsaufwand und kann das Verhalten in Richtung FCFS verschieben. Im Beispielszenario wird *P1* zunächst ausgeführt. Nach Ablauf des Quantums werden die bereiten Prozesse entsprechend der Warteschlangenreihenfolge berücksichtigt. Die Visualisierung muss deshalb Quantumablauf, Wiedereinreihung und Wechsel des laufenden Prozesses sichtbar machen.

2.5.3. LCFS

Bei Last-Come, First-Served (LCFS) wird der zuletzt eingetroffene bereite Prozess bevorzugt. In einer präemptiven Ausprägung verdrängt ein neuer Prozess den aktuell laufenden Prozess. Dadurch kann ein später eintreffender Prozess sehr schnell reagieren. Gleichzeitig besteht die Gefahr, dass ältere Prozesse wiederholt zurückgestellt werden und lange warten.

Für die Visualisierung sind bei LCFS insbesondere die Ankunftszeitpunkte, die Präempti-onsereignisse und die Rückkehr des verdrängten Prozesses in die Bereitschaftsmenge relevant. Die Anwendung stellt diese Vorgänge als zeitlich geordnete Ereignisse und Segmente dar.

2.5.4. Strict Priority

Strict Priority wählt aus den bereiten Prozessen denjenigen mit der höchsten Priorität. In der präemptiven Variante kann die Ankunft eines höher priorisierten Prozesses den laufenden Prozess unmittelbar verdrängen. Die Priorität muss dabei als Teil des Szenarios eindeutig definiert sein.

Die Stärke des Verfahrens liegt in der gezielten Bevorzugung wichtiger oder zeitkritischer Prozesse. Die zentrale Schwäche ist mögliche Starvation: Niedrig priorisierte Prozesse können trotz Bereitschaft lange nicht ausgeführt werden. Eine Analyse sollte deshalb neben Mittelwerten auch maximale Wartezeiten und die Ereignisfolge betrachten.

2.5.5. Multilevel Feedback Queue

Die Multilevel Feedback Queue (MLFQ) verwendet mehrere Bereitschaftwarteschlangen mit unterschiedlichen Prioritätsstufen und Zeitquantumswerten. Neue Prozesse werden

abhängig von der gewählten Strategie in eine Warteschlange eingeordnet. Verbraucht ein Prozess sein Quantum, ohne abzuschließen, kann er in eine niedrigere Warteschlange herabgestuft werden. Höhere Warteschlangen werden bevorzugt bedient; dadurch erhalten kurze oder interaktive Prozesse häufig eine schnelle erste Reaktion.

MLFQ ist adaptiv, weil die weitere Behandlung eines Prozesses von seinem bisherigen Verhalten abhängt. Die Strategie kann dadurch unterschiedliche Prozessarten ohne vorherige exakte Kenntnis ihrer gesamten Burst-Zeit behandeln. Gleichzeitig entstehen zusätzliche Zielkonflikte: Zu aggressive Bevorzugung hoher Ebenen kann Prozesse in niedrigen Ebenen benachteiligen, während lange Quantumswerte die Reaktionsfähigkeit verringern können. Aging oder periodische Promotionen sind typische Gegenmaßnahmen gegen Starvation, werden im fokussierten Modell jedoch nur berücksichtigt, wenn sie als Teil der konkreten Verfahrensvariante definiert sind.

Im Visualizer wird MLFQ durch mehrere Ebenen, Segmentpositionen, Quantumabläufe und Herabstufungsereignisse sichtbar gemacht. Dadurch lässt sich nicht nur das Endergebnis, sondern auch die Anpassung des Verfahrens an den bisherigen Prozessverlauf nachvollziehen.

2.6. Vergleich der theoretischen Perspektiven

Stallings sowie Tanenbaum und Bos behandeln beide Betriebssysteme, Prozesse und Scheduling, setzen aber unterschiedliche Akzente. Die Werke werden in dieser Arbeit nicht als widersprüchliche Theorien gegenübergestellt, sondern als sich ergänzende Zugänge verwendet.

Tabelle 2.3.: Ergänzende Perspektiven der verwendeten Betriebssystemliteratur.

Aspekt	Stallings	Tanenbaum und Bos
Grundperspektive	Systematische Betrachtung von Betriebssystemaufgaben, Ressourcen und Verwaltungszielen.	Konzeptionelle Betrachtung von Betriebssystemstrukturen, Prozessen, Threads und Abläufen.
Scheduling	Einordnung von Zielen und Kriterien der Prozessorzuteilung.	Erklärung von Prozessausführung, Präemption und dem Zusammenspiel der Betriebssystemkomponenten.
Nutzen für diese Arbeit	Begründung der Bewertungsmetriken und Zielkonflikte.	Begründung des Prozessmodells, der Zustandswechsel und der Ereignisorientierung.
Übertragung in den Visualizer	Vergleichsansicht mit Wartezeit, Durchlaufzeit, Reaktionszeit und Durchsatz.	Gantt-Segmente, Prozesszustände, Präemptionen und Kontextwechsel.

Stallings liefert damit vor allem den systematischen Rahmen für die Frage, nach welchen Kriterien Scheduling-Verfahren bewertet werden können. Tanenbaum und Bos ergänzen diesen Rahmen um die Perspektive auf Prozesse und die Abläufe, durch die Scheduling-Entscheidungen wirksam werden. Die Anwendung benötigt beide Ebenen: Ohne Metriken

wären Verfahren nicht vergleichbar; ohne Zustands- und Ereignismodell bliebe unklar, wie ein Ergebnis zustande gekommen ist Stallings [2003], Tanenbaum and Bos [2016].

2.7. Algorithmusvisualisierung als Vermittlungsform

Eine Scheduling-Visualisierung übersetzt abstrakte Zustandsänderungen in eine zeitlich beobachtbare Darstellung. Für die vorliegende Anwendung bedeutet dies, dass ein Prozess nicht nur als fertige Kennzahl, sondern als Folge von Ankunft, Bereitschaft, Ausführung, Präemption und Abschluss sichtbar wird. Die Gantt-Ansicht stellt die zeitliche Ordnung her, während Status- und Ereignisansichten die Bedeutung einzelner Abschnitte ergänzen.

Algorithmusvisualisierung ist dabei nicht automatisch mit Lernwirksamkeit gleichzusetzen. Die Meta-Studie von Hundhausen, Douglas und Stasko zeigt, dass der Nutzen algorithmischer Visualisierungen von Gestaltung, Aufgabenstellung und aktiver Auseinandersetzung abhängt Hundhausen et al. [2002]. Für diese Arbeit folgt daraus die Anforderung, Entscheidungswege und Interaktionsmöglichkeiten bereitzustellen. Eine empirische Untersuchung, ob Studierende dadurch tatsächlich besser lernen, ist ausdrücklich nicht Gegenstand der Arbeit.

2.8. Accessibility als fachliche und gestalterische Grundlage

Die fachliche Komplexität des Scheduling-Modells darf nicht dazu führen, dass zentrale Zustände ausschließlich visuell oder nur mit einer Maus zugänglich sind. Für die Anwendung sind daher semantische Struktur, Tastaturbedienung, sichtbarer Fokus, verständliche Statusmeldungen und eine nicht ausschließlich farbliche Unterscheidung relevant. Die WCAG 2.1 dient hierfür als normative Referenz W3C [2018]. Heinecke und Gerken ergänzen diese normative Perspektive um Grundlagen der Mensch-Computer-Interaktion und der Gestaltung verständlicher Interaktionen Heinecke and Gerken [2024].

Accessibility wird in dieser Arbeit nicht als nachträgliche Prüfung einzelner Farben verstanden. Sie betrifft auch die Modellierung der Zustände: Ein Dialog benötigt Fokusmanagement, ein Zeitregler eine verständliche Beschriftung und ein Gantt-Segment einen zugänglichen Namen. Diese Grundlagen führen direkt zu den in Kapitel 3 operationalisierten Anforderungen A11Y-01 bis A11Y-08.

2.9. Zwischenfazit

Das Kapitel hat das fachliche Modell für die folgenden Teile der Arbeit hergeleitet. Prozesse werden als zustandsbehaftete Ausführungseinheiten verstanden, deren CPU-Zuteilung durch deterministische Scheduling-Regeln gesteuert wird. Präemption, Kontextwechsel

und Ankunftsereignisse erzeugen eine zeitliche Folge, die über Kennzahlen und Visualisierung analysiert werden kann.

Für den Systementwurf ergeben sich daraus drei Konsequenzen: Erstens müssen Zustände, Ereignisse und Segmente auf einer gemeinsamen Datenbasis beruhen. Zweitens müssen mehrere Metriken parallel betrachtet werden, weil Scheduling-Ziele in Konkurrenz stehen. Drittens muss die Interaktion den Ablauf erklärbar und zugänglich machen. Diese Konsequenzen bilden die Brücke von den Grundlagen zur Anforderungsanalyse und zum Systementwurf.

3. Anforderungsanalyse

Die Anforderungen ergeben sich aus der Verbindung fachlicher Grundlagen des CPU-Scheduling mit didaktischen Zielen und softwaretechnischen Qualitätsansprüchen. Ziel ist eine Anwendung, die präemptive Verfahren korrekt abbildet und ihre Abläufe für Lernende verständlich zugänglich macht.

3.1. Zielgruppe und Einsatzkontext

Die primäre Zielgruppe sind Studierende in einführenden und vertiefenden Lehrveranstaltungen zu Betriebssystemen. Die Anwendung unterstützt sowohl die begleitete Demonstration im Unterricht als auch die eigenständige Analyse von Szenarien.

3.2. Fachliche Anforderungen

Die Anwendung muss die Verfahren Round Robin, LCFS, Strict Priority und MLFQ konsistent simulieren. Dafür sind zentrale Eingabeparameter wie Ankunftszeit, Burst-Zeit, Priorität und Zeitquantum korrekt zu verarbeiten. Als Ergebnis müssen insbesondere Wartezeit, Durchlaufzeit, Reaktionszeit, Kontextwechsel und Präemptionsanzahl bereitgestellt werden.

Eine Kernanforderung lautet: Simulationslogik, Ereignisverlauf und Kennzahlen müssen aus derselben Zustandsbasis abgeleitet werden, damit Ergebnisse fachlich konsistent und vergleichbar bleiben.

3.3. Didaktische Anforderungen

Die Anwendung soll nicht nur Endwerte liefern, sondern Entscheidungswege sichtbar machen. Lernende müssen erkennen können, warum Prozesse ausgewählt, unterbrochen oder zurückgestellt werden.

Im Zentrum stehen drei Leitprinzipien:

- **Transparenz:** Dispatch, Präemption, Kontextwechsel und Abschluss sind eindeutig darzustellen.

- **Vergleichbarkeit:** Mehrere Verfahren sollen auf identischer Szenariobasis gegenüberstellbar sein.
- **Kontrollierbares Lerntempo:** Schrittmodus und kontinuierliche Wiedergabe müssen unterstützt werden.

3.4. Interaktions- und Usability-Anforderungen

Die Oberfläche soll die kognitive Last reduzieren und den typischen Arbeitsablauf unterstützen: Szenario definieren, Verfahren auswählen, Lauf starten, Ergebnis interpretieren, Verfahren vergleichen. Funktionen müssen in konsistenter Reihenfolge verfügbar und ohne unnötige Bedienhürden nutzbar sein.

Für den Lehrkontext ist wesentlich, dass relevante Zustände unmittelbar erkennbar sind und die Bedienlogik die fachliche Analyse unterstützt statt überlagert.

3.5. Anforderungen an Accessibility

Da die Anwendung im Lehrkontext eingesetzt wird, sind grundlegende Accessibility-Anforderungen zu berücksichtigen: semantisch klare Struktur, nachvollziehbare Fokusführung, verständliche Statuskommunikation und ausreichende Kontraste. Ziel ist, dass zentrale Lern- und Bedienfunktionen auch ohne rein visuelle Orientierung nutzbar bleiben W3C [2018]. Die folgende Operationalisierung begrenzt den Prüfstand auf die tatsächlich relevante Scheduler-Oberfläche. Sie ist keine vollständige Konformitätserklärung.

Tabelle 3.1.: Operationalisierte Accessibility-Anforderungen des fokussierten Prüfstands.

ID	Anforderung	Referenz	Prüfverfahren und Grenze
A11Y-01	Semantische Struktur und verständliche Namen für Dialoge, Steuerelemente und Ansichten.	WCAG 1.3.1, 4.1.2	Statische Prüfung und manuelle Prüfung; keine vollständige AT-Kompatibilitätsmessung.
A11Y-02	Zentrale Funktionen sind vollständig per Tastatur bedienbar.	WCAG 2.1.1	Keyboard-Walkthrough; komplexe Browser-/Assistive-Tech-Kombinationen bleiben offen.
A11Y-03	Fokus ist sichtbar, logisch geführt und wird in Dialogen eingeschlossen und zurückgegeben.	WCAG 2.4.3, 2.4.7	Codeprüfung und manueller Dialogtest; keine formale Fokusabdeckungsmessung.
A11Y-04	Zustandsänderungen von Playback und Vergleich werden verständlich angekündigt.	WCAG 4.1.3	Prüfung der Statuskanäle und manueller Test; Sprachverständlichkeit wird nicht empirisch bewertet.
A11Y-05	Gantt-SVG und interaktive Segmente besitzen beschreibende Namen und Tastaturzugriff.	WCAG 1.1.1, 2.1.1	Statische Prüfung und manuelle Segmentprüfung; SVG wird nicht mit jedem Screenreader getestet.
A11Y-06	Farbe ist nicht alleiniger Informationsträger und Fokus-/Textkontraste sind ausreichend.	WCAG 1.4.1, 1.4.3, 1.4.11	Kontrastprüfung der verwendeten Zustände und manuelle Farbnutzung; frei kombinierbare Farben bleiben eine Grenze.
A11Y-07	Oberfläche bleibt bei Vergrößerung und schmalem Viewport nutzbar.	WCAG 1.4.4, 1.4.10	Prüfung bei 200 % und schmalem Viewport; keine Prüfung aller Endgeräte.
A11Y-08	Nicht notwendige Bewegung wird bei reduzierter Bewegungspräferenz vermieden.	WCAG 2.3.3	Medienpräferenz und Playback manuell prüfen; keine Aussage zu vestibulären Wirkungen.

WCAG 2.1 bildet dabei den normativen Maßstab. Das im Projektkontext verwendete Kurs- beziehungsweise Handoutmaterial dient als angewandte Entwurfsgrundlage, ersetzt aber weder die Primärquelle noch die hier vorgenommene Prüfung.

3.6. Technische Qualitätsziele

Die Anwendung soll modular, deterministisch und erweiterbar aufgebaut sein. Die Trennung von Simulationskern, Zustandsverwaltung und Visualisierung reduziert Fehleranfälligkeit und erleichtert Erweiterungen, etwa um weitere Algorithmen oder Analyseansichten.

Diese Qualitätsziele sichern sowohl Wartbarkeit als auch fachliche Verlässlichkeit im Lehrbetrieb.

3.7. Abgeleitete funktionale und nicht-funktionale Anforderungen

Die vorherigen Abschnitte lassen sich in funktionale und nicht-funktionale Kernanforderungen überführen.

Funktionale Anforderungen

- Erfassung und Bearbeitung von Prozessen mit relevanten Scheduling-Parametern.
- Simulation der Verfahren Round Robin, LCFS, Strict Priority und MLFQ.
- Visualisierung des zeitlichen Ablaufs inklusive Präemptionen und Zustandswechseln.
- Ausgabe zentraler Kennzahlen pro Lauf.
- Vergleich mehrerer Läufe auf identischer Szenariobasis.
- Interaktive Ablaufsteuerung mit Start, Pause, Schrittmodus und Zeitsprung.

Nicht-funktionale Anforderungen

- Deterministisches Verhalten bei identischer Eingabe.
- Klare Trennung von Fachlogik und Darstellung.
- Verständliche, konsistente und zugängliche Benutzerführung.
- Erweiterbarkeit für zusätzliche Verfahren und Darstellungsformen.
- Testbarkeit der Kernlogik und zentraler Interaktionspfade.

Die abgeleiteten Anforderungen bilden die Grundlage für den Systementwurf im folgenden Kapitel.

4. Systementwurf

Der Systementwurf folgt der Leitidee, fachliche Korrektheit und verständliche Interaktion gemeinsam umzusetzen. Ein Scheduling-Visualizer soll nicht nur Ergebnisse berechnen, sondern auch Entscheidungswege transparent machen. Entsprechend verbindet dieses Kapitel Architekturentscheidungen mit didaktischen Anforderungen.

4.1. Entwurfsziele und Leitprinzipien

Aus der Anforderungsanalyse ergeben sich vier zentrale Entwurfsziele:

- **Fachliche Konsistenz:** Simulation, Ereignisverlauf und Kennzahlen basieren auf einer gemeinsamen Zustandsbasis.
- **Didaktische Nachvollziehbarkeit:** Zustandswechsel und Präemptionen sind zeitlich eindeutig darzustellen.
- **Deterministik:** Identische Eingaben führen zu identischen Abläufen.
- **Erweiterbarkeit:** Neue Algorithmen und Darstellungsformen sollen ohne grundlegenden Umbau integrierbar sein.

Diese Ziele werden durch eine komponentenbasierte Struktur mit klaren Verantwortlichkeiten umgesetzt. Simulationslogik und Darstellung bleiben bewusst entkoppelt, sodass fachliche Änderungen kontrolliert möglich sind und die Oberfläche als Vermittlungsschicht wirken kann.

4.2. Gesamtarchitektur und Schichtenmodell

Das System ist in mehrere logisch getrennte, miteinander verbundene Bereiche gegliedert:

- **Präsentationsschicht:** Darstellung der Fokusansicht, der Vergleichsübersicht, der Gantt-Zeitachse, der Statusansicht und der Bewertungsmatrix.
- **Anwendungslogik:** Wiederverwendbare Composables für Szenarioverwaltung, Vergleich und Wiedergabesteuerung.
- **Simulationskern:** Deterministische Berechnung des zeitlichen Ablaufs durch die Scheduling Engine.

- **Domänenmodell:** Gemeinsame fachliche Datenbasis aus Szenario und Simulation-Run.

Die konkrete Komponenten- und Datenstruktur ist in Abbildung 4.1 dargestellt. Die Anwendungshülle `AppShell.vue` vermittelt zwischen Benutzerinteraktion, Composables und den sichtbaren Ansichten. Die Steuerung ist damit nicht in einer eigenständigen fachlichen Zustandswelt gekapselt, sondern wirkt über die Anwendungslogik auf die gemeinsame Datenbasis ein.

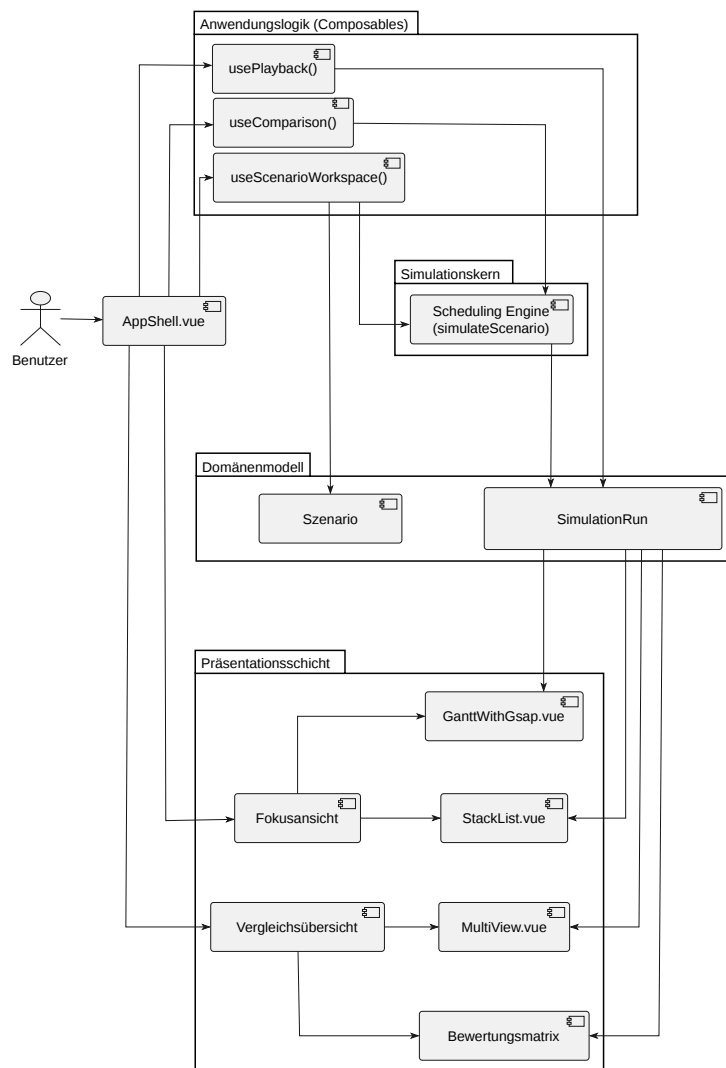


Abbildung 4.1.: Komponentenmodell der Anwendung mit Präsentationsschicht, Composables, Simulationskern und Domänenmodell.

Abbildung 4.1 zeigt den zentralen Daten- und Kontrollfluss der Anwendung. Die Benut-

zerinteraktion wird zunächst von `AppShell.vue` aufgenommen und an die zuständigen Composables weitergeleitet. `useScenarioWorkspace()` verwaltet Szenarien und stößt die Berechnung eines Laufs an, während `usePlayback()` die Navigation durch die erzeugten Zustandsabbilder steuert und `useComparison()` den Vergleich mehrerer Läufe unterstützt.

Die Scheduling Engine mit `simulateScenario()` verarbeitet das Szenario und erzeugt ein `SimulationRun`. Dieses Ergebnisobjekt bildet die gemeinsame fachliche Grundlage für die Präsentationsschicht. Die Fokusansicht nutzt daraus unter anderem die Gantt-Darstellung durch `GanttWithGsap.vue` und die Statusdarstellung durch `StackList.vue`. Die Vergleichsübersicht verwendet `MultiView.vue` und die Bewertungsmatrix, um mehrere erzeugte Läufe gegenüberzustellen.

Die Architektur trennt damit die Berechnung eines Ablaufs von seiner Darstellung, ohne den Datenfluss unnötig zu vervielfachen. Neue Ansichten können auf den bestehenden Ergebnisstrukturen aufbauen, während Änderungen an der Scheduling Engine nicht unmittelbar eine Anpassung jeder einzelnen Präsentationskomponente erfordern.

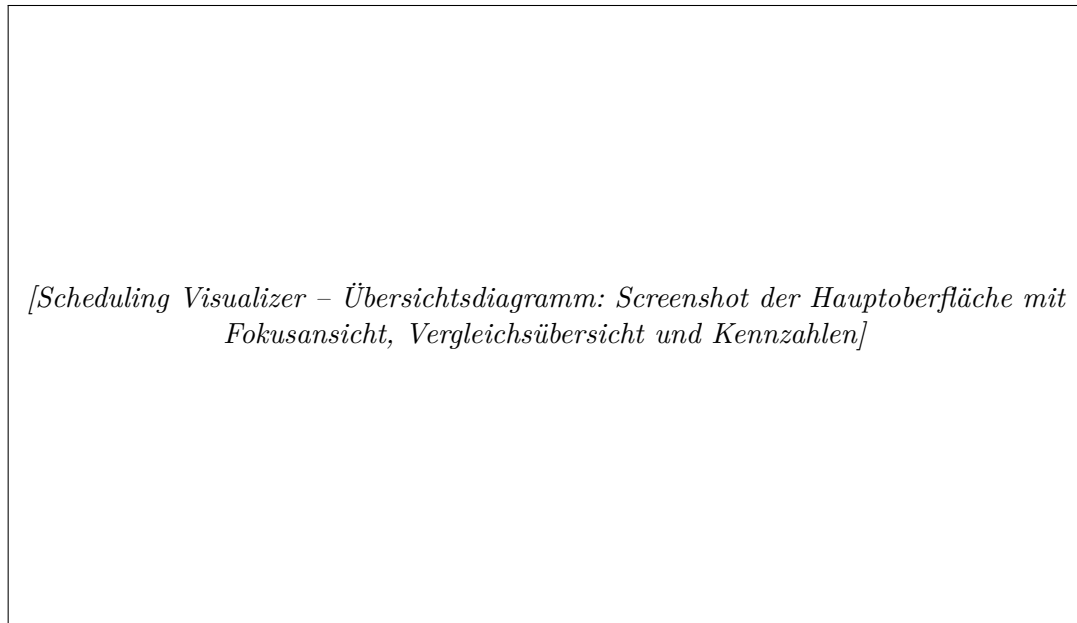


Abbildung 4.2.: Übersicht der Benutzeroberfläche: Eingabebereich, Simulationsfenster und Metriken-Anzeige.

4.3. Komponenten und Verantwortlichkeiten

4.3.1. Szenario- und Konfigurationskomponente

Diese Komponente verwaltet die Eingabedaten der Prozesse (z. B. Ankunftszeit, Burst-Zeit, Priorität) sowie algorithmusspezifische Parameter (z. B. Zeitquantum). Zusätzlich übernimmt sie die Validierung und Normalisierung der Eingaben, sodass alle Läufe auf strukturell konsistenten Daten aufsetzen.

4.3.2. Simulationskomponente

Die Simulationskomponente bildet den fachlichen Kern. Aus Szenario und Verfahrensdefinition erzeugt sie den vollständigen Ablauf als deterministische Folge von Zustandsübergängen. Ergebnisse werden nicht nur als Endzustand abgelegt, sondern in auswertbaren Strukturen bereitgestellt (Segmente, Ereignisse, Snapshots, Kennzahlen).

4.3.3. Visualisierungs- und Interaktionskomponente

Diese Komponente überführt Simulationsdaten in Analyseansichten: zeitlicher Ablauf (Gantt), Warteschlangen-/Statusdarstellung und Kennzahlen. Gleichzeitig steuert sie die Wiedergabe über Interaktionsfunktionen und ermöglicht den Vergleich mehrerer Läufe auf identischer Datenbasis.

4.4. Datenfluss und Zustandsmodell

Der Datenfluss folgt einem festen Muster:

1. Eingabeszenario wird erfasst und validiert.
2. Der Simulationskern berechnet den Ablauf für ein Verfahren.
3. Ergebnisstrukturen (Ereignisse, Segmente, Snapshots, Metriken) werden bereitgestellt.
4. Die Darstellungsschicht rendert die aktuelle Sicht und reagiert auf Interaktionsbefehle.

Entscheidend ist, dass Visualisierung und Kennzahlen auf denselben Ergebnisstrukturen basieren. Dadurch entstehen keine widersprüchlichen Anzeigen zwischen Ablaufdarstellung und numerischer Bewertung.

4.5. Interaktionsmodell für den Lernprozess

Das Interaktionsmodell folgt einem didaktisch motivierten Nutzungsfluss:

1. Szenario definieren,
2. Verfahren auswählen,
3. Ablauf steuern und beobachten,
4. Ergebnisse interpretieren,
5. Verfahren vergleichen.

Die Interaktion ist damit kein rein technischer Bedienvorgang, sondern Teil des Lernpro-

zesses. Die Steuerung beeinflusst die Sicht auf den Ablauf (z. B. Schrittmodus), ohne die fachliche Berechnung selbst zu verändern.

4.6. HCI-Leitlinien im Entwurf

Für eine Lernanwendung ist die Benutzeroberfläche ein fachlich relevantes Element. Der Entwurf folgt daher grundlegenden HCI-Leitlinien:

- zentrale Zustände müssen direkt sichtbar sein,
- wichtige Ereignisse (Präemption, Kontextwechsel, Abschluss) müssen klar markiert werden,
- Bedienung und Analyse sollen in einem konsistenten Ablauf zusammenwirken,
- die Oberfläche soll Orientierung bieten und zusätzliche kognitive Last minimieren.

Damit wird sichergestellt, dass die Anwendung nicht nur technisch korrekt arbeitet, sondern auch als didaktisches Werkzeug nutzbar ist.

4.7. Accessibility als Entwurfsprinzip

Accessibility wird im Entwurf als Querschnittseigenschaft der Interaktions- und Darstellungsschicht behandelt. Dialoge besitzen einen eindeutig benannten Zustand, einen initialen Fokus und eine definierte Rückgabe an das auslösende Element. Das Composable `useFocusTrap` kapselt diese Logik, sodass Welcome-, Auswahl-, Generator- und Ranking-Dialog nach demselben Muster bedient werden.

Die Gantt-Ansicht wird als strukturiertes SVG mit Titel, Beschreibung und Zusammenfassung ausgegeben. Interaktive Segmente erhalten zusätzlich eigene zugängliche Namen und können per Enter oder Leertaste aktiviert werden. Die Vergleichsansicht verwendet eine echte Tabstruktur mit ausgewähltem Tab, zugehörigem Panel und Pfeiltastennavigation. Statusänderungen werden über separate Statuskanäle angekündigt, damit sie nicht ausschließlich über Farbe oder Bewegung vermittelt werden.

Farbe dient der visuellen Orientierung, ist jedoch nicht die einzige Unterscheidung. Aktive, beste und schwächste Zustände erhalten zusätzlich Text oder strukturelle Merkmale. Fokusindikatoren, Forced-Colors-Regeln, Beschriftungen für Zeitsteuerungen und die Reaktion auf reduzierte Bewegung sind Teil des Entwurfs. Diese Entscheidungen adressieren die A11Y-Anforderungen aus Tabelle 3.1; sie begründen jedoch keine vollständige WCAG-Konformität.

4.8. Erweiterbarkeit und Entwurfsgrenzen

Die Architektur ist auf Erweiterbarkeit ausgelegt: Neue Scheduling-Verfahren können als zusätzliche Strategien im Simulationskern ergänzt werden, ohne die UI grundlegend umzubauen. Ebenso lassen sich neue Kennzahlen oder Ansichten integrieren, sofern sie auf den vorhandenen Ergebnisstrukturen aufsetzen.

Eine bewusste Entwurfsgrenze liegt in der Fokussierung auf präemptive Verfahren und den Lehrkontext. Die Architektur ist zwar übertragbar, aber nicht als vollständiger Ersatz für produktive Betriebssystem-Scheduler konzipiert.

5. Implementierung

Die Implementierung überführt den in Kapitel 4 beschriebenen Entwurf in eine lauffähige Webanwendung. Ziel ist die technisch konsistente Realisierung einer Visualisierungsanwendung, die präemptive Scheduling-Verfahren korrekt berechnet und verständlich darstellt.

5.1. Technische Grundlagen und verwendete Technologien

Die Anwendung wurde als clientseitige Webanwendung mit Vue 3 und TypeScript umgesetzt. Vue.js Core Team [2026]. Vue strukturiert die Benutzeroberfläche komponentenbasiert, während TypeScript die fachlichen Datenmodelle typensicher abbildet. Der Build- und Entwicklungsprozess erfolgt mit Vite [2026], die automatisierte Verifikation mit Vitest Vitest Team [2026]. Für gezielte Übergänge in der Oberfläche wird GSAP eingesetzt GreenSock [2026].

Eine zentrale technische Entscheidung ist die Trennung von Simulationslogik, Zustandsverwaltung und Darstellung. Dadurch bleibt der Simulationskern unabhängig testbar, während die Oberfläche auf konsistenten Ergebnisdaten operiert.

5.1.1. Begründung der Technologieentscheidungen

Die Auswahl des Technologie-Stacks orientiert sich an den fachlichen und didaktischen Anforderungen der Anwendung. Da der Scheduling-Visualizer vollständig clientseitig arbeitet und unmittelbar auf Benutzerinteraktionen reagieren soll, wurde eine Webtechnologie gewählt, die komponentenbasierte Oberflächen, reaktive Zustandsaktualisierung und eine einfache Bereitstellung als statische Anwendung verbindet.

Vue 3 wurde als UI-Framework gewählt, weil die Oberfläche in klar abgegrenzte Komponenten für Szenariokonfiguration, Laufsteuerung, Visualisierung und Vergleichsansicht aufgeteilt werden kann. Vue.js Core Team [2026]. Die reaktive Aktualisierung sorgt dafür, dass Änderungen am aktiven Laufzustand direkt in den betroffenen Ansichten sichtbar werden. Die Composition API unterstützt dabei die Trennung wiederverwendbarer Zustands- und Interaktionslogik von der konkreten Darstellung. Dadurch lässt sich beispielsweise die Wiedergabesteuerung unabhängig von einer einzelnen Ansicht organisieren.

TypeScript ergänzt diese Struktur um statische Typisierung. Domänenobjekte wie Prozesse, Ereignisse, Segmente, Snapshots und Metriken bilden Schnittstellen zwischen Simu-

lationskern und Benutzeroberfläche. Eine explizite Typisierung macht diese Schnittstellen bereits während der Entwicklung überprüfbar und reduziert das Risiko, dass eine Ansicht unvollständige oder strukturell falsche Simulationsergebnisse verarbeitet. Dies unterstützt insbesondere das Prinzip der gemeinsamen Ergebnisbasis, das mit `SimulationRun` umgesetzt wird.

Vite wurde als Build- und Entwicklungswerkzeug eingesetzt Vite [2026]. Für die Entwicklung ist vor allem die kurze Rückkopplungszeit durch schnelle Aktualisierung relevant: Änderungen an Komponenten und Styles können unmittelbar geprüft werden. Für die Bereitstellung erzeugt Vite zudem statische Artefakte, die anschließend unabhängig von einer serverseitigen Anwendungslogik ausgeliefert werden können.

Für die Gantt-Darstellung und weitere zeitbezogene Ansichten wird SVG verwendet. SVG eignet sich für die präzise Positionierung von Zeitabschnitten, Segmentgrenzen, Beschriftungen und Ereignismarkierungen. Die einzelnen Elemente bleiben dabei im Dokumentmodell adressierbar und können mit CSS oder Ereignisbehandlung verbunden werden. Gegenüber einer reinen Canvas-Darstellung erleichtert dies die Zuordnung von Interaktionen und Beschriftungen zu konkreten Prozessen.

GSAP ergänzt die Darstellung für gezielte Übergänge und Animationen GreenSock [2026]. Die Bibliothek wird bewusst nur für Präsentationseffekte eingesetzt; sie verändert weder Scheduling-Entscheidungen noch die erzeugten Ergebnisdaten. Damit bleibt die fachliche Berechnung deterministisch, während Zustandswechsel in der Oberfläche zeitlich nachvollziehbar hervorgehoben werden können.

Vitest wurde für die automatisierte Verifikation der Simulationslogik gewählt Vitest Team [2026]. Die deterministische Ausführung der Verfahren ermöglicht reproduzierbare Tests von Ereignisfolgen, Präemptionen, Segmenten und Kennzahlen. Die Tests greifen dabei auf die fachlichen Ergebnisstrukturen zu und müssen die Darstellung nicht nachbilden. Dadurch werden fachliche Fehler unabhängig von Layout- oder Animationseffekten erkennbar.

Die Technologieentscheidungen bilden somit eine zusammenhängende Architektur: Vue strukturiert die Interaktion, TypeScript beschreibt die fachlichen Verträge, der Simulationskern erzeugt reproduzierbare Daten, SVG stellt diese präzise dar und Vitest überprüft die Kernlogik. Vite und GSAP unterstützen Entwicklungsprozess und Präsentation, ohne die fachliche Zuständigkeit des Simulationskerns zu erweitern.

5.2. Datenmodell und Simulationskern

Die fachliche Basis bildet ein einheitliches Prozessmodell mit Attributen wie Ankunftszeit, Burst-Zeit, Priorität, Restzeit und Status. Auf dieser Grundlage werden alle Verfahren innerhalb eines gemeinsamen Simulationsrahmens berechnet.

Ein Simulationslauf erzeugt mehrere zusammenhängende Ergebnisstrukturen:

- **Ereignislog:** chronologische Ereignisse (z. B. Dispatch, Präemption, Abschluss),
- **Segmente:** zeitlich abgegrenzte Ausführungsintervalle für die Gantt-Darstellung,
- **Snapshots:** Zustandsabbilder für Wiedergabe und Zeitsprung,
- **Metriken:** aggregierte Kennzahlen wie Warte-, Durchlauf- und Reaktionszeit sowie Kontextwechsel.

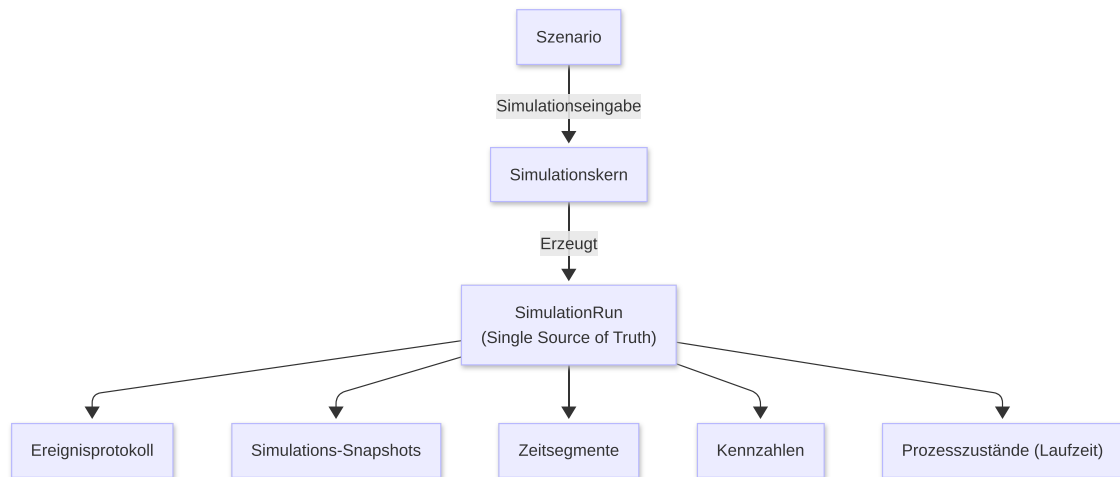


Abbildung 5.1.: Domänenmodell des Simulationslaufs mit seinen Ergebnisstrukturen.

Das Domänenmodell zeigt **SimulationRun** als zentrales Ergebnisobjekt. Es bündelt Ereignisse, Snapshots, Segmente, Kennzahlen und Prozesszustände und bildet damit die gemeinsame Datenbasis für nachgelagerte Ansichten.

5.2.1. Single Source of Truth: SimulationRun

Die Implementierung folgt dem Prinzip der *Single Source of Truth*: **SimulationRun** repräsentiert das vollständige Ergebnis eines Simulationsdurchlaufs. Nachgelagerte Darstellungen – insbesondere Gantt-Visualisierung, Vergleichsansichten und Kennzahlen – greifen ausschließlich auf dieses Objekt zu. Dadurch basieren Darstellung und Bewertung auf identischer Datenbasis.

SimulationRun umfasst den Ereignisverlauf (**events**), Zustandsabbilder (**snapshots**), Ausführungssegmente (**segments**), aggregierte Kennzahlen (**finalMetrics**) und Prozesszustände (**processStates**). Inkonsistenzen zwischen visualisiertem Ablauf und numerischer Auswertung werden dadurch vermieden.

Die zentrale Ergebnisstruktur unterstützt zugleich Reproduzierbarkeit und Testbarkeit: Bei identischem Szenario und identischen Parametern entsteht deterministisch dieselbe Struktur; Tests können direkt auf Ereignisse, Segmente und Kennzahlen zugreifen. Neue Ansichten lassen sich ergänzen, ohne den Simulationskern zu verändern.

5.3. Umsetzung der Scheduling-Verfahren

Round Robin, LCFS, Strict Priority und MLFQ wurden als präemptive Strategien innerhalb derselben Simulationslogik implementiert:

- **Round Robin:** Präemption nach Ablauf eines konfigurierbaren Zeitquantums.
- **LCFS (präemptiv):** Neu eintreffende Prozesse verdrängen den laufenden Prozess.
- **Strict Priority:** Auswahl des höchstpriorisierten bereiten Prozesses mit sofortiger Präemption bei höher priorisierten Neuankünften.
- **MLFQ:** Mehrere Warteschlangen mit stufenabhängigen Quantumswerten und Herabstufungslogik.

Alle Verfahren nutzen dieselbe Tick- und Ereignisverarbeitung. Unterschiede im Ablauf und in den Kennzahlen ergeben sich damit aus der Verfahrensstrategie, nicht aus unterschiedlicher Berechnungsinfrastruktur.

5.3.1. Ablauf eines Simulationslaufs

`simulateScenario()` erhält das konfigurierte Szenario, führt die tick-basierte Simulation aus und erzeugt ein vollständiges `SimulationRun`. Die Oberfläche rendert anschließend die enthaltenen Segmente, Snapshots, Ereignisse und Kennzahlen.

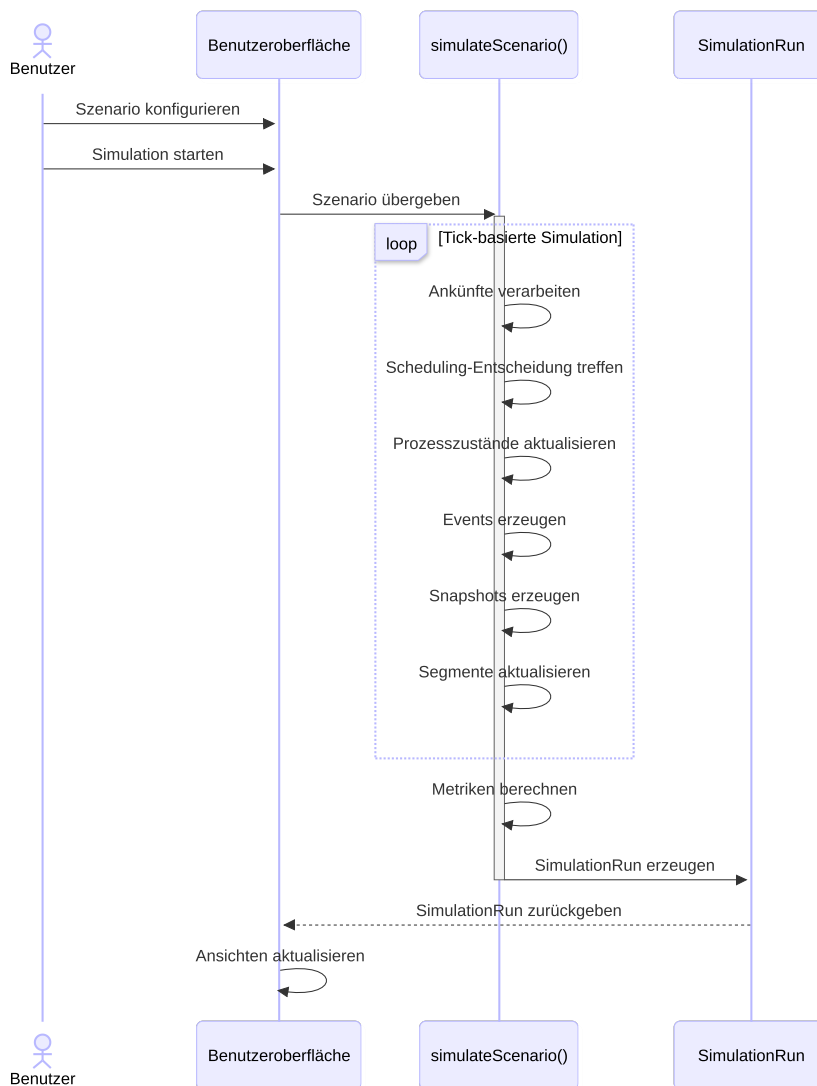


Abbildung 5.2.: Sequenzdiagramm eines Simulationslaufs von der Benutzeraktion bis zur Aktualisierung der Benutzeroberfläche.

Die tick-basierte Schleife verarbeitet Ankünfte, Scheduling-Entscheidungen, Präemptionen und Abschlüsse in zeitlicher Reihenfolge. Die erforderlichen Artefakte für Wiedergabe und Auswertung entstehen innerhalb dieses Ablaufs, sodass die Oberfläche auf einem vorberechneten Laufzustand navigieren kann.

5.3.2. Prozesszustände und Übergänge

Jeder Prozess durchläuft einen definierten Lebenszyklus. Die Zustände **pending**, **ready**, **running**, **preempted** und **finished** beschreiben, ob ein Prozess noch nicht eingetroffen, zur Ausführung bereit, aktiv, verdrängt oder abgeschlossen ist.

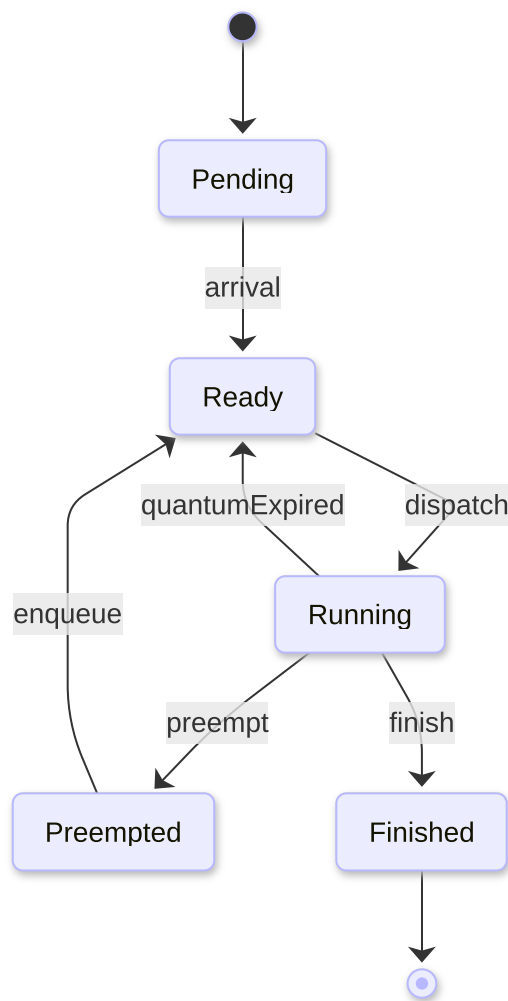


Abbildung 5.3.: Zustände und Übergänge eines Prozesses im Simulationsmodell.

Die Übergänge sind mit Ereignissen verknüpft: **arrival** (Eintritt), **dispatch** (CPU-Zuteilung), **preempt** (Verdrängung), **quantumExpired** (Ablauf des Zeitquantums) und **finish** (Abschluss). Die code-nahe Benennung erleichtert die Rückverfolgbarkeit zwischen Modell, Implementierung und Tests.

5.4. Umsetzung der Visualisierung

Die Visualisierung ist in getrennte Funktionsbereiche gegliedert: Eingabe und Konfiguration, Laufsteuerung, Fokusansicht, Kennzahlenbereich und Vergleichsansicht.

Die zeitliche Ausführung wird als Gantt-Darstellung auf Basis der Segmentdaten visualisiert. Ergänzende Ansichten zeigen Wartesituationen und laufzeitbezogene Kennzahlen. Die Umsetzung nutzt SVG-basierte Darstellungen, um Zeitachsen, Segmentgrenzen und Ereignisse präzise abzubilden.

Ziel ist fachliche Lesbarkeit: Entscheidungen des Schedulers sollen zeitlich und inhaltlich

nachvollziehbar werden.

5.5. Interaktionsmechanismen und Nutzerworkflow

Die Interaktion folgt einem didaktischen Ablauf: Szenario definieren, Verfahren auswählen, Lauf steuern, Ergebnis interpretieren und Verfahren vergleichen.

Die Wiedergabelogik arbeitet auf vorberechneten Snapshots und startet die Simulation nicht bei jedem Interaktionsschritt neu. Dadurch bleiben Funktionen wie Start, Pause, Schrittmodus, Geschwindigkeitsanpassung und Zeitsprung reaktiv, ohne die Konsistenz der Ergebnisdaten zu gefährden.

Im Vergleichsmodus werden mehrere Läufe auf identischer Eingabebasis nebeneinander dargestellt. Das unterstützt die direkte Gegenüberstellung algorithmischer Unterschiede.

5.6. Umsetzung der Accessibility-Anforderungen

Die Dialogkomponenten verwenden mit `useFocusTrap` eine gemeinsame Implementierung für Escape, Tab-Zirkulation, initialen Fokus, `aria-hidden/inert` auf dem Hauptinhalt und Fokus-Rückgabe. Der Ranking-Dialog wurde an dieses Muster angeschlossen. Dadurch wird der Dialog nicht nur visuell überlagert, sondern auch als modal begrenzter Interaktionszustand behandelt.

Die Vergleichsansicht in `ComparisonPanel.vue` ist als Tablist mit `role=tab`, `aria-selected`, `aria-controls` und Tabpanels umgesetzt. Pfeil-, Home- und End-Tasten verschieben den Fokus innerhalb der Auswahl. In `GanttWithGsap.vue` und `Scrubber.vue` sind die Zeitregler sichtbar beschriftet und mit dem live aktualisierten Zeitstatus verbunden.

`GanttWithGsap.vue` ist die im `AppShell` verwendete produktive Gantt-Komponente. Sie bündelt Playback, Slider, Statuskommunikation, GSAP-Übergänge und die Reduced-Motion-Präferenz. Für die eigentliche SVG-Struktur rendert sie intern die spezialisierte Komponente `Gantt.vue`. Dort werden Titel, Beschreibung, Zusammenfassung und zugängliche Namen der einzelnen Prozesssegmente erzeugt. Bei geteilten Segmenten bleibt nur ein Teil ein eigenständiges Tastaturziel, damit dieselbe fachliche Information nicht doppelt angeboten wird. Das Burger-Menü unterstützt Escape, Erstfokus und Fokus-Rückgabe. Nicht notwendige GSAP- und Hover-Effekte werden bei aktivierter Präferenz für reduzierte Bewegung übersprungen; Änderungen dieser Präferenz werden während der Laufzeit berücksichtigt.

5.7. Build- und Deploymentprozess

Für die Bereitstellung wird ein containerbasierter Deploymentprozess verwendet. Die Anwendung wird mit Vite in statische Artefakte überführt und in ein Docker-Image paketierte. Dadurch entsteht eine reproduzierbare Laufzeitumgebung unabhängig vom lokalen Entwicklungsrechner.

Das Deployment erfolgt auf einer Hosting-Plattform mit Container-Unterstützung. Änderungen können konsistent gebaut und in stabiler Umgebung veröffentlicht werden; im Lehrkontext stellt dies eine einheitliche Versionsbasis für Demonstration und Nutzung sicher.

5.8. Besondere technische Herausforderungen

Eine zentrale Herausforderung war die Synchronisation von Simulationsgenauigkeit und interaktiver Bedienung. Präemptive Verfahren erzeugen häufige Zustandswechsel, während die Oberfläche gleichzeitig unmittelbar auf Interaktionen reagieren muss.

Ein weiterer Schwerpunkt war die Sicherung der Deterministik durch klare Zustandsübergänge, konsistente Datenmodelle und testbare Schnittstellen. Zusätzlich ergibt sich ein Zielkonflikt zwischen Detailtiefe und Ressourcenbedarf: Dichtere Snapshot-Folgen verbessern Analyse und Zeitsprung, erhöhen jedoch Speicher- und Verarbeitungsaufwand. Dieser Konflikt wurde durch parametrisierbare Granularität der Wiedergabedaten adressiert.

6. Qualitätssicherung

Für eine didaktisch ausgerichtete Anwendung mit fachlichem Anspruch ist Qualitätssicherung ein zentraler Entwicklungsbestandteil. Bewertet werden nicht nur technische Funktionsfähigkeit, sondern insbesondere Konsistenz der Simulationslogik, Reproduzierbarkeit der Ergebnisse und Verlässlichkeit der Interaktion.

6.1. Qualitätsziele und Prüfstrategie

Die Qualitätssicherung verfolgt drei Hauptziele:

- **Fachliche Korrektheit:** Scheduling-Entscheidungen und Kennzahlen müssen konsistent abgeleitet werden.
- **Reproduzierbarkeit:** Identische Eingaben müssen identische Abläufe erzeugen.
- **Bedienbarkeit und Zugänglichkeit:** Zentrale Funktionen müssen verlässlich und verständlich nutzbar sein.

Zur Erreichung dieser Ziele wurde ein mehrstufiges Vorgehen gewählt: automatisierte Tests der Kernlogik, Prüfung zentraler Interaktionspfade sowie Bewertung grundlegender Accessibility-Aspekte.

6.2. Automatisierte Verifikation der Simulationslogik

Die Simulationslogik wird mit Vitest automatisiert geprüft Vitest Team [2026]. Im Fokus stehen deterministische Berechnungsschritte, korrekte Präemptionsentscheidungen und konsistente Kennzahlenableitung.

Geprüft werden unter anderem:

- Quantum-Verhalten und Wiedereinreihung bei Round Robin,
- Präemption bei Neuankunft in präemptiven Verfahren (z. B. LCFS, Strict Priority),
- Herabstufungslogik in MLFQ,
- Konsistenz von Ereignisfolge, Segmenten und Endzustand.

Zusätzlich wird validiert, ob unvollständige bzw. ungültige Eingaben robust normalisiert werden und erzeugte Segmente gültige Zeitgrenzen aufweisen.

6.3. Prüfung von Interaktion und Accessibility

Ergänzend zur Fachlogik wurde die Oberfläche hinsichtlich Bedienbarkeit und Zugänglichkeit geprüft. Betrachtet wurden die in Tabelle 3.1 definierten A11Y-Anforderungen. Als Referenz dient WCAG 2.1 W3C [2018]; die Prüfung ist auf den Scheduler fokussiert und keine vollständige Konformitätsbewertung.

Für den Lehrkontext ist wesentlich, dass Benutzerinnen und Benutzer ohne zusätzliche Hürden zwischen Szenariokonfiguration, Ablaufsteuerung und Ergebnisinterpretation wechseln können. Klare Statusanzeigen und eindeutige Steuerzustände unterstützen diese Nutzbarkeit.

6.4. Reproduzierbarkeit und Vergleichbarkeit

Ein zentrales Qualitätsmerkmal ist die deterministische Reproduzierbarkeit: Bei identischem Szenario und identischen Parametern entsteht derselbe Ablauf mit denselben Kennzahlen. Diese Eigenschaft ist für belastbare Vergleichsvergleiche wesentlich, da Unterschiede eindeutig auf die jeweilige Scheduling-Strategie zurückgeführt werden können.

Auch die Vergleichsansicht profitiert davon: Mehrere Läufe lassen sich auf gemeinsamer Eingabebasis direkt gegenüberstellen, ohne dass zufällige Abweichungen die Interpretation verzerren.

6.4.1. Automatisierte und statische Prüfungen

Die automatisierten Tests werden mit Vitest ausgeführt. Im aktuellen Stand bestehen zehn Testdateien mit 35 Tests. Davon prüfen die beiden Accessibility-Dateien insgesamt elf statische Marker- und Strukturbedingungen. Sie lesen Quelltext und prüfen beispielsweise Dialogrollen, Fokusmanagement, Tabsemantik, Sliderbeschriftungen, Gantt-Beschreibungen und Reduced-Motion-Gating. Der Test mit dem Dateinamen `axe.runtime.test.ts` führt keinen DOM-Scan mit `axe-core` aus und wird deshalb nicht als axe-Runtime-Nachweis gezählt. Der vorhandene Placeholder-Test bleibt als Scaffold erhalten und wird ebenfalls nicht als Qualitätsnachweis gezählt.

6.4.2. Manuelles Prüfprotokoll

Die manuelle Prüfung umfasst einen Keyboard-only-Durchlauf, Dialogöffnung und Escape, Fokusfalle und Fokus-Rückgabe, Burger-Menü, Tab-Navigation, Slider, Playback, Gantt-Segmente und Statusmeldungen. Zusätzlich werden Desktop- und schmale Viewports, 200

6.5. Erreichte Testabdeckung und Grenzen

Zum Zeitpunkt der Prüfung umfasst die automatisierte Testsuite zehn Testdateien mit insgesamt 35 erfolgreichen Tests. Damit werden zentrale Kern- und Integrationspfade sowie statische Accessibility-Merkmale abgedeckt und die technische Verlässlichkeit der Implementierung erhöht.

Die Testsuite stellt jedoch keinen vollständigen formalen Korrektheitsbeweis für alle denkbaren Prozesskonstellationen und keinen vollständigen WCAG-/BITV-Audit dar. Ebenso ersetzt sie keine empirische Untersuchung der Lernwirksamkeit mit Nutzergruppen. Die Qualitätssicherung ist daher als begrenzter, risikoorientierter Prüfstand zentraler Funktionen zu verstehen.

6.6. Fazit zur Qualitätssicherung

Qualitätssicherung wurde als integraler Bestandteil der Entwicklung umgesetzt. Die Kombination aus deterministischer Simulationslogik, automatisierter Verifikation und strukturierter Interaktionsprüfung bildet die Grundlage für ein fachlich belastbares und didaktisch nutzbare Lehrwerkzeug.

7. Ergebnis und Produktpräsentation

Dieses Kapitel stellt das entwickelte Produkt vor und ordnet es anhand der zuvor definierten Ziele ein. Im Fokus stehen Funktionsumfang, Nutzungsablauf und fachliche Belastbarkeit der Anwendung als didaktisches Werkzeug.

7.1. Entwickeltes Endprodukt

Das Ergebnis der Arbeit ist eine interaktive Webanwendung zur Visualisierung präemptiver CPU-Scheduling-Verfahren. Sie ermöglicht die Definition von Prozessen mit relevanten Parametern, die Ausführung unterschiedlicher Verfahren und die vergleichende Analyse von Ablauf und Kennzahlen.

Das Produkt vereint drei Kerneigenschaften:

- konsistente Simulationslogik,
- interaktive und verständliche Ablaufdarstellung,
- direkte Vergleichbarkeit mehrerer Verfahren auf identischer Datenbasis.

Damit geht die Anwendung über eine reine Demonstration hinaus und ist als Werkzeug für Lehr- und Lernszenarien einsetzbar.

7.2. Produktüberblick und Nutzerfluss

Der typische Nutzungsablauf folgt einer festen Sequenz:

1. Definition oder Auswahl eines Szenarios,
2. Auswahl und Parametrisierung eines Scheduling-Verfahrens,
3. Ausführung und Steuerung des Laufs,
4. Interpretation von Ablauf und Kennzahlen,
5. Vergleich mit alternativen Verfahren.

Dieser Ablauf unterstützt das didaktische Ziel, Entscheidungen nicht nur als Endwerte, sondern im zeitlichen Kontext ihrer Entstehung zu analysieren.

7.3. Produktpräsentation mit Screenshots

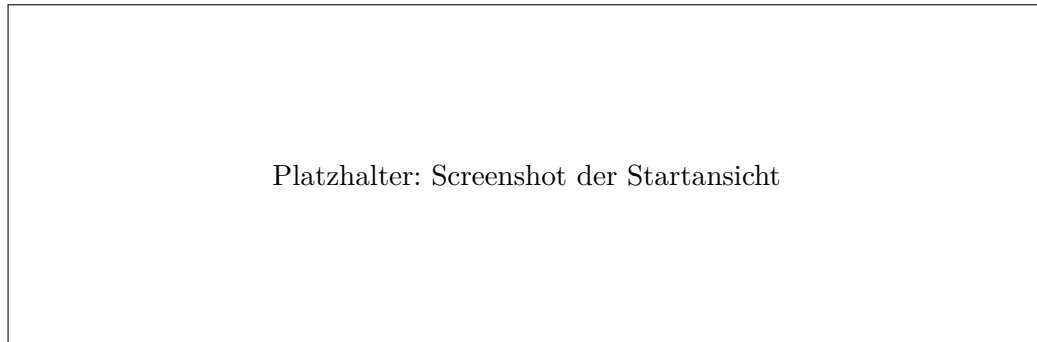


Abbildung 7.1.: Startansicht der Anwendung mit Ausgangszustand und Einstieg in die Szenario-konfiguration.

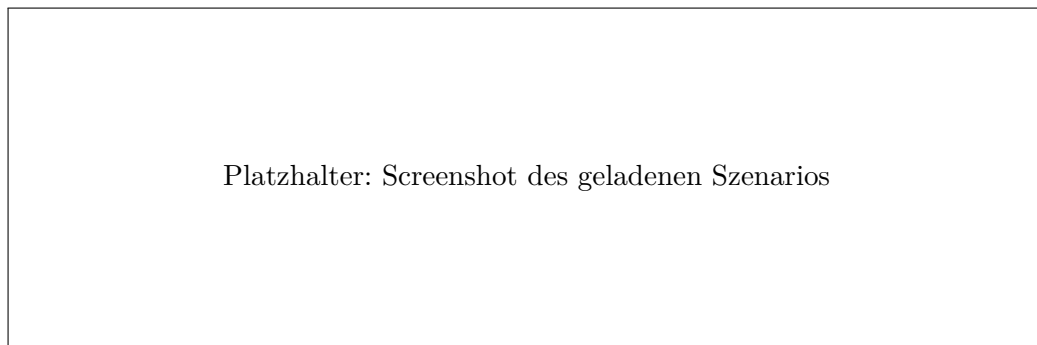


Abbildung 7.2.: Geladenes Szenario mit sichtbaren Task-Parametern als Grundlage für die Simulation.

7.4. Umgesetzte Funktionalität im Soll-Ist-Abgleich

Die aus Kapitel 3 abgeleiteten Anforderungen wurden in den zentralen Funktionsbereichen umgesetzt:

- **Szenarioverwaltung:** Erfassung und Bearbeitung von Task-Daten mit relevanten Scheduling-Parametern.
- **Verfahrensauswahl:** Ausführung von Round Robin, LCFS, Strict Priority und MLFQ innerhalb eines einheitlichen Simulationsrahmens.
- **Ablaufsteuerung:** Start, Pause, Schrittmodus, Zeitsprung und Geschwindigkeitsanpassung.
- **Visualisierung:** Zeitliche Darstellung der Ausführung mit Ereignisbezug.
- **Kennzahlenanalyse:** Ausgabe zentraler Metriken pro Lauf.
- **Vergleichsmodus:** Gegenüberstellung mehrerer Läufe unter identischen Eingaben.

Platzhalter: Screenshot der Fokusansicht

Abbildung 7.3.: Fokusansicht mit zeitlicher Ablaufdarstellung und Kennzahlenbereich für einen einzelnen Lauf.

Platzhalter: Detailansicht des Gantt-Diagramms

Abbildung 7.4.: Detaildarstellung der Gantt-Ansicht mit segmentierter Ausführung und sichtbaren Task-Wechseln.

Platzhalter: Warteschlangen- und Statusdarstellung

Abbildung 7.5.: Warteschlangen- und Statusdarstellung zur Einordnung der aktuellen Scheduling-Entscheidung.

Platzhalter: Detailansicht der Kennzahlen

Abbildung 7.6.: Kennzahlenbereich mit Wartezeit, Durchlaufzeit und Reaktionszeit als quantitative Auswertung des Laufs.



Platzhalter: Vergleichsansicht mehrerer Verfahren

Abbildung 7.7.: Vergleichsansicht mehrerer Verfahren auf identischer Szenariobasis.

Damit ist der funktionale Kern vollständig umgesetzt. Nicht-funktionale Ziele wie Deterministik, Nachvollziehbarkeit und Erweiterbarkeit werden durch Architektur und Implementierung unterstützt.

7.5. Accessibility-relevante Produktzustände

Die Produktnachweise sind nicht nur als visuelle Screenshots zu verstehen. Die Fokusansicht unterstützt die Handlung, den Ablauf über Start, Pause, Schrittsteuerung und Zeitregler zu untersuchen; der Regler besitzt eine sichtbare Beschriftung und einen angekündigten Zeitstatus. Der Gantt-Nachweis zeigt Prozesssegmente, Zeitgrenzen und Ereignismarkierungen. Ein Segment kann per Tastatur fokussiert und aktiviert werden, sodass die Ablaufanalyse nicht ausschließlich von Pointer-Interaktion abhängt.

Der Dialognachweis steht für einen begrenzten Interaktionszustand: Beim Öffnen wird der Fokus in den Dialog geführt, Tab bleibt innerhalb des Dialogs und Escape schließt ihn. Nach dem Schließen kehrt der Fokus zum auslösenden Element zurück. Die Vergleichsansicht stellt Anwendungsfälle als Tabs dar; der aktive Zustand wird nicht nur farblich, sondern auch über Auswahlstatus und zugehöriges Panel ausgedrückt. Statusmeldungen zu bestem und schwächstem Lauf werden zusätzlich über einen Statuskanal bereitgestellt.

Diese Zustände belegen konkrete Nutzerhandlungen und Implementierungsentscheidungen. Sie sind zugleich nur ein fokussierter Nachweis für die in Tabelle 3.1 definierten Anforderungen und keine Aussage über vollständige WCAG- oder BITV-Erfüllung.

7.6. Ergebnisbetrachtung

Das entwickelte Produkt erfüllt die Zielsetzung der Arbeit: Präemptive Scheduling-Verfahren werden nicht nur berechnet, sondern als dynamischer Ablauf verständlich und vergleichbar dargestellt. Die Verknüpfung von Ablaufvisualisierung, Kennzahlen und Interaktion ermöglicht die Analyse algorithmischer Unterschiede in einem didaktisch nutzbaren Kontext.

Besonders relevant ist die Sichtbarkeit der Entscheidungspfade. Benutzerinnen und Benutzer erkennen nicht nur das Ergebnis, sondern auch dessen Entstehung im zeitlichen Verlauf. Daraus ergibt sich ein praktischer Mehrwert für Lehre, Demonstration und eigenständiges Lernen.

8. Fazit und Ausblick

Die vorliegende Arbeit zeigt, dass sich präemptive CPU-Scheduling-Verfahren in einer interaktiven Webanwendung fachlich konsistent und didaktisch wirksam aufbereiten lassen. Die zentrale Leistung liegt in der Verbindung von Simulationslogik, Visualisierung und steuerbarer Interaktion zu einem integrierten Lehrwerkzeug.

Ausgehend von den theoretischen Grundlagen wurden Anforderungen an Fachlogik, Usability und Transparenz abgeleitet und in ein umsetzbares Architekturmodell überführt. Die Implementierung realisiert diese Anforderungen in einer modularen Struktur mit klarer Trennung von Simulationskern und Darstellung. Dadurch bleiben Berechnungen nachvollziehbar, testbar und unter identischen Bedingungen reproduzierbar.

Die Anwendung erfüllt damit die Zielsetzung der Arbeit: Scheduling-Entscheidungen werden sichtbar, zeitliche Abläufe mit Kennzahlen verknüpft und Verfahren auf gemeinsamer Szenariobasis direkt vergleichbar gemacht. Der didaktische Mehrwert liegt insbesondere in der Darstellung von Entscheidungspfaden statt reiner Endergebnisse.

Die Qualitätssicherung stützt diese Einordnung durch automatisierte Tests der Kernlogik, die Prüfung zentraler Interaktionspfade sowie die Berücksichtigung grundlegender Accessibility-Anforderungen. Damit ist die Anwendung sowohl als technische Umsetzung als auch als Lerninstrument belastbar einsetzbar.

8.1. Ausblick

Für die Weiterentwicklung ergeben sich mehrere Richtungen. Erstens kann die fachliche Breite durch zusätzliche Scheduling-Verfahren und differenziertere Lastmodelle erweitert werden. Zweitens lässt sich die Analyse durch weitere Kennzahlen, Exportfunktionen und ergänzende Vergleichsmodi vertiefen. Drittens sind didaktische Erweiterungen wie geführte Lernpfade, Aufgabenserien und vorbereitete Szenariosets für den Einsatz in Lehrveranstaltungen sinnvoll.

Darüber hinaus ist eine empirische Evaluation mit Studierenden ein naheliegender nächster Schritt, um den Lerneffekt systematisch zu untersuchen. Die vorliegende Arbeit schafft dafür eine tragfähige technische und konzeptionelle Grundlage.

Literaturverzeichnis

GreenSock. Gsap documentation, 2026. URL <https://greensock.com/docs/>.

Andreas M. Heinecke and Jens Gerken. *Mensch-Computer-Interaktion: Basiswissen für Entwickler und Gestalter*. Xpert.press. Springer Vieweg, Berlin/Heidelberg, 3 edition, 2024. ISBN 978-3-662-67568-7.

Christopher D. Hundhausen, Sarah A. Douglas, and John T. Stasko. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages & Computing*, 13(3): 259–290, 2002. doi: 10.1006/jvlc.2002.0237.

William Stallings. *Betriebssysteme: Prinzipien und Umsetzung*. Pearson Studium, München, 4., überarb. auflage edition, 2003.

Andrew S. Tanenbaum and Herbert Bos. *Moderne Betriebssysteme*. Pearson Studium, München, 4., aktualisierte auflage edition, 2016.

Vite. Vite documentation, 2026. URL <https://vitejs.dev/>.

Vitest Team. Vitest documentation, 2026. URL <https://vitest.dev/>.

Vue.js Core Team. Vue 3 documentation, 2026. URL <https://vuejs.org/>.

W3C. Web content accessibility guidelines (wcag) 2.1, 2018. URL <https://www.w3.org/TR/WCAG21/>.

A. Anhang

A.1. Ergänzende Abbildungen

A.2. Zusätzliche Testfälle und Szenarien

A.3. Operationalisierungsmatrix Accessibility

Die vollständige Matrix ergänzt die Kurzfassung aus Tabelle 3.1 um Implementierungsort, Nachweis und Ergebnisstatus.

ID	Entwurfsentscheidung	Implementierung/Nachweis	Ergebnis und Grenze
A11Y-01	Semantische Dialoge, Namen und Panels.	Dialogkomponenten, Tablist, Statusrollen; statischer Test.	Erfüllt im fokussierten Umfang; Screenreader-Matrix offen.
A11Y-02	Keyboard-first-Bedienung.	Menü, Tabs, Gantt und Playback; manueller Walkthrough.	Geprüft; Browser-/Assistive-Tech-Kombinationen begrenzt.
A11Y-03	Sichtbarer Fokus und Dialogfalle.	<code>useFocusTrap</code> , Ranking-Dialog; Code- und manueller Test.	Erfüllt für die geprüften Dialogpfade.
A11Y-04	Statuskanäle statt ausschließlich visueller Effekte.	Live-Status in Playback und Vergleich; manueller Test.	Erfüllt im geprüften Ablauf.
A11Y-05	Beschreibbares, interaktives SVG.	<code>Gantt.vue</code> , Segmentlabels; statisch und manuell.	Erfüllt im fokussierten SVG; keine vollständige AT-Abdeckung.
A11Y-06	Nichtfarbliche Zustandsmerkmale und Kontrastprüfung.	Statuslabels, Fokusregeln, <code>Forced Colors</code> ; manueller Checker.	Zustände geprüft; freie Prozessfarben bleiben Grenze.
A11Y-07	Reflow und Zoom.	Schmalen Viewport und 200 % Zoom; manuelles Protokoll.	Ergebnis nach Durchführung eintragen; keine Geräteabdeckung.
A11Y-08	Reduzierte Bewegung.	Medienpräferenz in <code>GanttWithGsap.vue</code> ; manueller Test.	Nicht notwendige Effekte reduziert; keine Nutzerstudie.

A.4. Kompaktes manuelles Accessibility-Prüfprotokoll

Setup	Prüfschritte	Beobachtung/Status
Desktop, Tastatur	Dialog öffnen, Tab/Shift-Tab, Escape, erneut fokussieren.	Fokusfalle und Rückgabe dokumentieren: behoben/offen.
Desktop, Tastatur	Menü, Tabs, Slider, Playback und Gantt ohne Maus bedienen.	Reihenfolge, Aktivierung und Status dokumentieren.
Schmalere Viewport, 200 %	Vergleich, Gantt und Steuerung bei vergrößerter Darstellung prüfen.	Überlappung, Abschneiden und Reflow dokumentieren.
Reduced Motion/Forced Colors	Präferenz aktivieren und Zustände erneut ausführen.	Nicht notwendige Bewegung und Fokuskontrast dokumentieren.
Kontrastprüfung	Tatsächlich verwendete Text-, Fokus- und Statuszustände messen.	Messwert und verbleibende Farbrisiken dokumentieren.

A.5. Ergänzende technische Artefakte (z. B. Konfigurationsbeispiele)

Eidesstattliche Erklärung

Hiermit versichere ich, dass ich diese BACHELORARBEIT selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, alle Ausführungen, die anderen Schriften wörtlich oder sinngemäß entnommen wurden, kenntlich gemacht sind und die Arbeit in gleicher oder ähnlicher Fassung noch nicht Bestandteil einer Studien- oder Prüfungsleistung war.

Marten Kirschenhofer